

Little Einstein

FESMARO 2025



Nama Anggota:

Akbar Zaidan Rohman

Austin Kane (**Ketua**)

Dave Eugene Wijaya

Kategori Lomba:

Analisis Big Data

Nama Institusi:

Universitas Bina Nusantara

Repository GitHub: <https://github.com/Tinnne/SeleksiFESMARO.git>

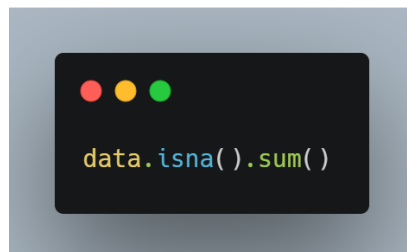
Dataset 1 - Amazon Reviews

Di era digital, ulasan pelanggan menjadi salah satu faktor krusial yang tidak boleh dipandang sebelah mata. Banyak perusahaan melakukan analisis sentimen untuk mencari tahu respon pelanggan terhadap produk/ jasa mereka. Hasil analisis inilah yang kemudian akan menjadi insight untuk mengambil keputusan yang lebih baik kedepannya. Namun, nyatanya faktor *volume* dan *velocity* dari ulasan pengguna begitu besar dan cepat yang membuat bisnis susah menganalisis dan belajar dari ulasan tersebut.

Analisis ini bertujuan untuk memahami pola sentimen pengguna Amazon terhadap produk-produk yang dibelinya, serta mengklasifikasikan review ke dalam penilaian positif atau negatif.

Dataset dan Pengelolaan Data

- Berikut adalah beberapa atribut dari dataset “Dataset Pertama_train.csv”
 - Label: Angka 1 menunjukkan ulasan negatif, angka 2 menunjukkan ulasan positif
 - Title: Berisikan judul dari review pelanggan
 - Text: Berisikan isi review pelanggan
- Exploratory Data Analysis
 - Pada dataset pertama, kami menemukan 207 row data title null, yang kedepannya akan ditangani pada saat data cleaning.



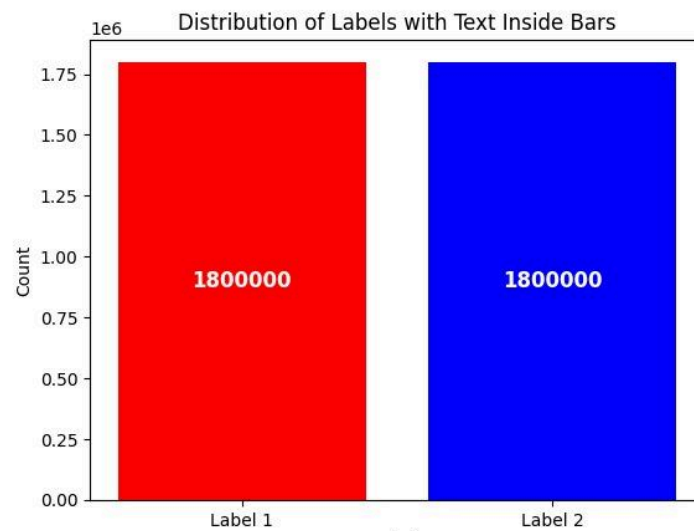
```
label    0
title    207
text     0
dtype: int64
```

- Pada saat di cek, data yang diberikan tidak memiliki data yang duplikat

```
data.duplicated().sum()
```

```
np.int64(0)
```

- Selain itu juga dapat dilihat bahwa jumlah dari masing-masing label sudah seimbang.



- Data cleaning
 - Karena ada data title yang null, maka kami memilih untuk mengganti value null tersebut dengan string kosong, karena nanti kedepannya model machine learning akan dilatih menggunakan title + text.

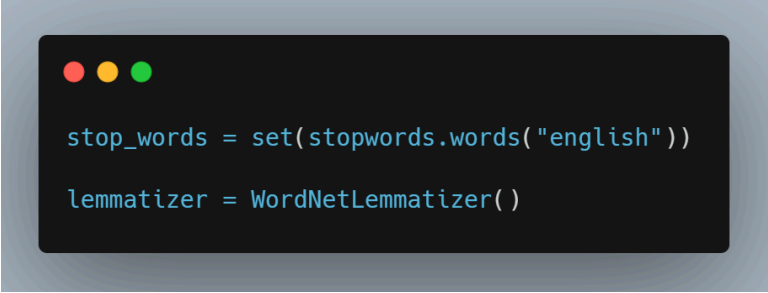
```
data = data.filled(" ")
```

```
label    0
title    0
text     0
dtype: int64
```

- Data Preprocessing

- Stopwords, mencakup kata-kata seperti “this”, “is”, “an”, “the”, “of”. Dengan menghapus stopwords, dari dataset kita, maka kita bisa memperpendek jumlah kata yang harus dianalisis oleh model. Cth: Kalimat “This is a very good product” akan terpotong menjadi “very good product”. Kedua kalimat itu memiliki arti yang sama, tetapi kalimat yang pendek akan lebih mudah dianalisis oleh model.

Lemmatizer berfungsi untuk mengubah tulisan ke dalam bentuk asalnya. Contoh: Buying -> Buy, Running -> Run. Dengan adanya perubahan ini, model akan lebih mudah memahami arti dari kata-kata di dalam kalimat ini. Hal ini juga memastikan adanya konsistensi dalam penggunaan kata, agar tidak membuat model bingung.



```
stop_words = set(stopwords.words("english"))  
lemmatizer = WordNetLemmatizer()
```

- Contractions bertujuan untuk mengubah kata-kata singkatan menjadi tidak dalam bentuk singkatan. Cth: I’m -> I am, Don’t -> Do not. Hal ini akan mempermudah model untuk mengerti dataset kita dengan lebih baik. Hal ini juga membantu adanya konsistensi agar model tidak semakin bingung dengan *slang* penulisan review orang yang berbeda-beda.



```
def expand_contractions(text):  
    return contractions.fix(text)
```

- Menggunakan NLTK untuk memecah kalimat menjadi kata-kata tersendiri melalui word_tokenize function. Selain itu, NLTK juga digunakan untuk menghapus stopwords dari text tersebut. Hal ini dilakukan untuk mempermudah machine learning nantinya.

```
def preprocess_text(text):
    tokens = word_tokenize(text.lower())
    tokens = [word for word in tokens if word.isalnum()] # Remove punctuations in the text
    tokens = [word for word in tokens if word not in stop_words] # Remove all stopwords from the text
    return " ".join(tokens)

data['combined_text'] = (data['title'] + " " + data['text']).apply(preprocess_text)
```

- Negations, function handle negations dibuat agar kata-kata negations seperti “not”, “no”, dst tidak terhapuskan dan dipisah menggunakan _. Hal ini bertujuan untuk mempermudah model mengerti kalimat yang sedang dianalisis.

```
def handle_negations(words):
    negation_words = {"not", "no", "never", "n't"}
    processed_words = []

    i = 0
    while i < len(words):
        if words[i] in negation_words and i+1 < len(words):
            processed_words.append(words[i] + "_" + words[i+1])
            i+=2
        else:
            processed_words.append(words[i])
            i+= 1
    return processed_words
```

- Function preprocess, merupakan sebuah function yang akan dijalankan untuk setiap kata di dalam kalimat yang ingin modelnya analisis. Pada function ini kita akan membuat semua tulisan lowercase, menghapus tanda baca, memanggil function handle negation and menggunakan lemmatize yang sudah kita declare di atas.

```
def preprocess_text(text):
    tokens = word_tokenize(text.lower())
    text = re.sub(r'\d+', '', text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    words = word_tokenize(text)
    words = handle_negations(words)
    words = [word for word in words if word not in stop_words]
    tokens = [lemmatizer.lemmatize(word) for word in words]
    return " ".join(words)
```

- Mengaplikasikan seluruh function yang sudah dibuat ke dalam text kita. Dalam kode dibawah, kita menggabungkan kolom title dan text ke dalam variable `combined_text` untuk dianalisis. Setelah itu apply function `expand_contractions` dan `preprocess_text` ke dalam text.

```
data['combined_text'] = (data['title'] + " " + data['text'])
data["combined_text"] = data["combined_text"].apply(expand_contractions)
data["combined_text"] = data["combined_text"].apply(preprocess_text)
```

- Menggunakan `TfidfVectorizer` pada data untuk mengkonversi data yang berupa tulisan menjadi bentuk angka agar bisa dipahami saat machine learning. Parameter `ngrams` menyatakan berapa banyak word sequence yang ingin kita buat, dengan parameter `ngram 1,2` maka *unigrams* dan *bigrams* akan dikonsiderasikan oleh vectorizernya. Contoh
 - Kalimat: Produk gelas ini bagus
 - Unigrams: ["Produk", "gelas", "ini", "bagus"]
 - Bigrams: ["Produk gelas", "ini bagus"]
 Selain itu, parameter `sublinear_tf` membantu menghilangkan bias dengan cara menghambat kata-kata yang sering muncul untuk terlalu mendominasi modelnya.

```
vectorizer = TfidfVectorizer(  
    ngram_range=(1,2),  
    max_features=5000,  
    sublinear_tf=True  
)  
  
X = vectorizer.fit_transform(data["combined_text"])  
y = data["label"]
```

Metode Analisis & Alat Teknologi

- Menggunakan metode eksplorasi data (EDA) serta model machine learning logistic regression untuk melakukan prediksi ulasan termasuk positif atau negatif.
- Alat & teknologi
 - VsCode: Aplikasi yang digunakan sebagai *code editor*
 - Anaconda: Software yang digunakan untuk membuat virtual environment
 - Google Drive: Untuk mengekstrak dataset dan mengimpornya ke dalam VsCode
 - Python: Bahasa pemrograman utama
 - Pandas & NumPy: Pengelolaan dan manipulasi data
 - Scikit-Learn: Machine learning dan evaluasi model
- Alasan pemilihan metode
 - Logistic regression digunakan karena memiliki performa yang baik untuk memprediksi apakah ulasan tersebut terklasifikasi ke dalam ulasan positif atau negatif.

Modeling

- Data dibagi menjadi 80% training dan 20% testing dengan random state sebesar 42. Hal tersebut kami lakukan karena pada Google Drive yang diberikan hanya terdapat train dataset dan tidak ada test datasetnya.



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42, stratify=y)
```

- Model machine learning yang digunakan adalah logistic regression. Hal ini dikarenakan logistic regression merupakan model yang bagus untuk melakukan binary classification, yakni apakah review tersebut positif atau negatif



```
params = {  
    'C': [0.01, 0.1, 1, 10],  
    'penalty': ['l1', 'l2'],  
    'solver': ['liblinear', 'saga']  
}  
  
clf = LogisticRegression(max_iter=5000)  
grid_search = HalvingGridSearchCV(clf, params, cv=5, scoring='accuracy', n_jobs=2)  
grid_search.fit(X_train, y_train)  
  
print("Best Parameter: ", grid_search.best_params_)
```

```
Best Parameter: {'C': 1, 'penalty': 'l1', 'solver': 'saga'}
```



```

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {accuracy: .4f}")
print("Classification Report: \n", classification_report(y_test, y_pred))
print("Confusion Matrix: \n" ,cm)

```

- **Metode evaluasi**

- Akurasi: 91.6%
- Confusion Matrix
 - True Positive: 329636
 - False Positive: 30123
 - False Negative: 30364
 - True Negative: 329877
- Classification Report
 - Precision: 92%
 - Recall: 92%
 - F1-Score: 92%
 - Support: 360.000

```

Accuracy: 0.9160
Classification Report:

```

	precision	recall	f1-score	support
1	0.92	0.92	0.92	360000
2	0.92	0.92	0.92	360000
accuracy			0.92	720000
macro avg	0.92	0.92	0.92	720000
weighted avg	0.92	0.92	0.92	720000

```

Confusion Matrix:
[[329636 30364]
 [ 30123 329877]]

```

Hasil dan Analisis

- Akurasi model mencapai 91.6%, yang menunjukkan bahwa model berhasil secara benar mengklasifikasikan 91.6% dari keseluruhan data.
- Precision (0,92): Dari semua prediksi yang dibuat untuk suatu kelas, 92% benar.
- Recall (0,92): Model berhasil mengidentifikasi 92% dari seluruh instance yang benar-benar termasuk dalam setiap kelas.
- F1-score (0,92): Rata-rata harmonik dari precision dan recall, menunjukkan keseimbangan performa model.
- Macro avg (0,92): Rata-rata F1-score dari semua kelas tanpa mempertimbangkan jumlah sampel di setiap kelas.
- Weighted avg (0,92): F1-score yang dihitung dengan mempertimbangkan jumlah sampel di setiap kelas.
- Confusion Matrix
 - True Positives (TP) / Prediksi Benar:
 - Kelas 1: 329.636 diklasifikasikan dengan benar sebagai kelas 1.
 - Kelas 2: 329.877 diklasifikasikan dengan benar sebagai kelas 2.
 - False Positives (FP) / Salah Prediksi:
 - 30.123 data kelas 1 salah diprediksi sebagai kelas 2.
 - 30.364 data kelas 2 salah diprediksi sebagai kelas 1.

Kesimpulan dan Rekomendasi

- Model logistic regression memberikan akurasi sebanyak 91.6% dalam melakukan klasifikasi sebuah review termasuk positif atau negatif. Akurasi menunjukkan angka yang cukup tinggi. Dengan ini perusahaan dapat dengan mudah mengklasifikasi review dan testimoni dan mengetahui bagaimana pandangan konsumen terhadap produk mereka. Saran untuk pengembangan lebih lanjut adalah mencoba jenis model yang lain selain logistic regression, karena beberapa model bekerja lebih baik dengan dataset tertentu.

Lampiran

- Sumber data:  Big Data

Dataset 2 - DataCo Smart Supply Chain

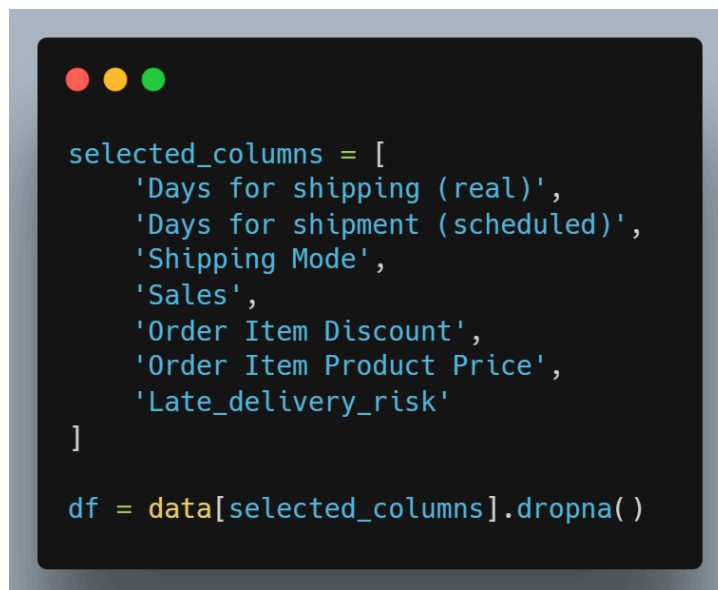
Pengiriman barang tepat waktu merupakan faktor kepuasan pelanggan. Oleh karena itu, analisis faktor-faktor yang mempengaruhi risiko keterlambatan pengiriman sangat penting untuk meningkatkan kenyamanan pelanggan

Analisis ini bertujuan untuk memahami faktor yang mempengaruhi risiko keterlambatan pengiriman berdasarkan data *history*. Model prediksi dibuat untuk mengklasifikasikan apakah suatu pengiriman memiliki risiko keterlambatan atau tidak

Analisis ini berfokus pada dataset **DataCo Smart Supply Chain** (Dataset Kedua) dengan menggunakan metode *data exploration*, *data cleaning*, *feature transformation*, serta pemodelan dengan algoritma machine learning untuk memprediksi risiko keterlambatan pengiriman.

Dataset dan Pengelolaan Data

- Berikut beberapa atribut yang digunakan dari dataset "Dataset Kedua.csv":
 - Days for shipping: Jumlah hari aktual untuk pengiriman
 - Days for shipment: Jumlah hari yang dijadwalkan untuk pengiriman
 - Shipping Mode: Mode pengiriman yang dipilih pelanggan
 - Sales: Total penjualan
 - Order Item Discount: Diskon yang diberikan pada item
 - Order Item Product Price: Harga produk
 - Late_delivery_risk: Indikator risiko keterlambatan pengiriman



```
selected_columns = [  
    'Days for shipping (real)',  
    'Days for shipment (scheduled)',  
    'Shipping Mode',  
    'Sales',  
    'Order Item Discount',  
    'Order Item Product Price',  
    'Late_delivery_risk'  
]  
  
df = data[selected_columns].dropna()
```

- Data cleaning
 - Menghapus kolom yang tidak relevan, seperti:
 - Customer Email,
 - Customer Password, dan
 - Product Image.
 - Menghapus duplikasi data
 - Mengubah format tanggal menjadi tipe datetime (untuk analisis waktu)

```
# Data Cleaning
irrelevant_cols = ['Customer Email', 'Customer Password', 'Product Image']
data.drop(columns=[col for col in irrelevant_cols if col in data.columns], inplace=True)
data.drop_duplicates(inplace=True)
data.fillna(method='ffill', inplace=True)

date_cols = ['order date (DateOrders)', 'shipping date (DateOrders)']
for col in date_cols:
    if col in data.columns:
        data[col] = pd.to_datetime(data[col])
```

- Transformasi data
 - Menggunakan Label Encoding pada kolom Shipping Mode untuk mengonversi kategori menjadi nilai numerik

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Shipping Mode'] = le.fit_transform(df['Shipping Mode'])
```

- Melakukan standarisasi distribusi fitur menggunakan StandardScaler untuk memastikan distribusi data seragam dalam model machine learning.

```
from sklearn.model_selection import train_test_split

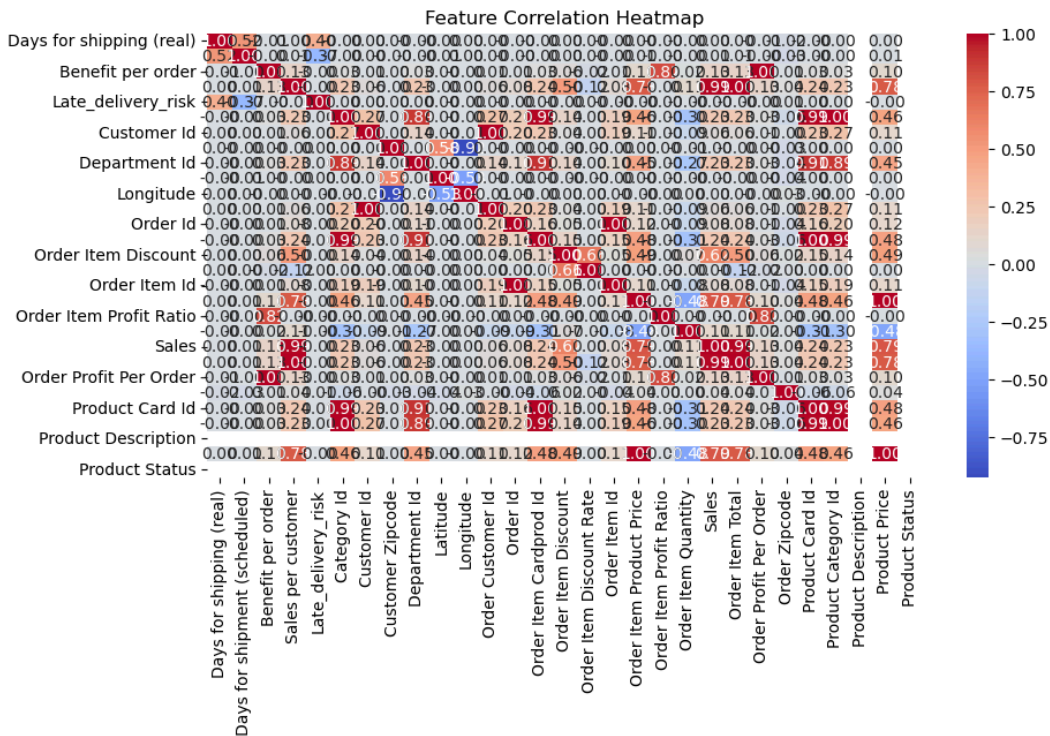
X = df.drop(columns=['Late_delivery_risk'])
y = df['Late_delivery_risk']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Metode Analisis & Alat Teknologi

- Menggunakan metode eksplorasi data (EDA) serta model machine learning Random Forest untuk klasifikasi risiko keterlambatan pengiriman
- Tools Teknologi
 - VsCode: Aplikasi yang digunakan sebagai *code editor*
 - Anaconda: Software yang digunakan untuk membuat virtual environment
 - Google Drive: Untuk mengekstrak dataset dan mengimpornya ke dalam VsCode
 - Python: Bahasa pemrograman utama
 - Pandas & NumPy: Pengelolaan dan manipulasi data
 - Scikit-Learn: Machine learning dan evaluasi model
- Alasan pemilihan metode
 - Random Forest dipilih karena memiliki performa yang baik dalam menangani dataset dengan fitur numerik dan kategorikal
 - GridSearchCV digunakan untuk mengoptimalkan parameter model

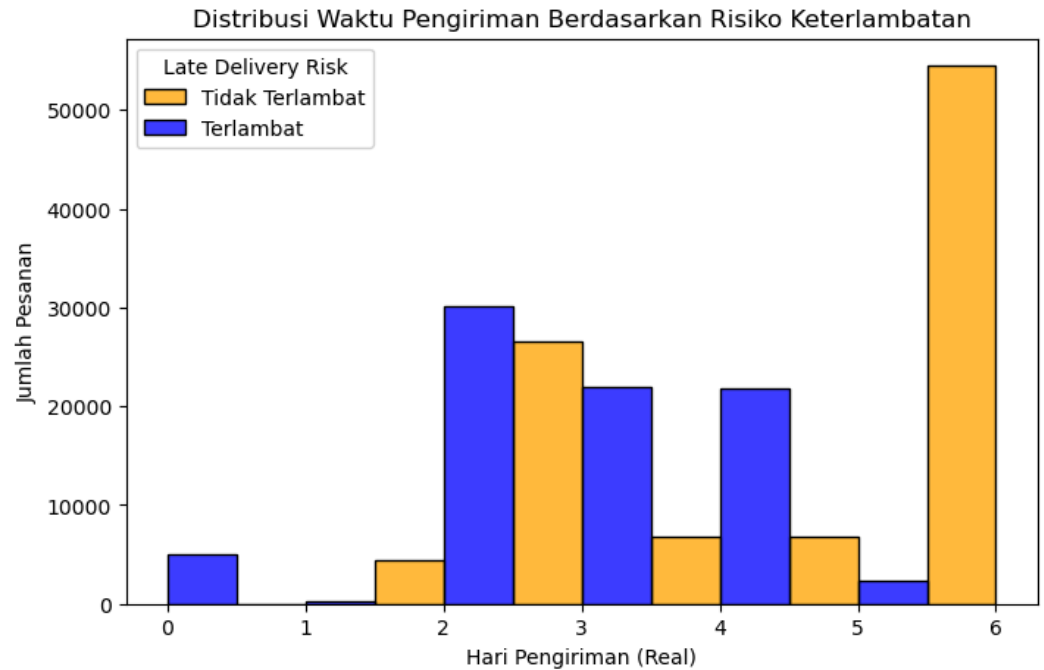
Exploratory Data Analysis (EDA)

- Visualisasi Data dengan menggunakan Heatmap



- Visualisasi data

- Histogram untuk distribusi **Days for shipping (real)** terhadap **Late_delivery_risk**



- Countplot untuk perbandingan **Shipping Mode** dengan **Late_delivery_risk**



- Insight yg didapat
 - Semakin besar selisih antara hari pengiriman dan yang dijadwalkan, semakin tinggi resiko keterlambatan (korelasi negatif)
 - Shipping Mode memiliki pengaruh yang lumayan terhadap keterlambatan pengiriman, jadi antara Shipping Mode dengan keterlambatan pengiriman mempunyai kuat

Modeling

- Data dibagi menjadi **80% training** dan **20% testing** dengan stratifikasi berdasarkan target **Late_delivery_risk**

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

- Algoritma menggunakan **Random Forest Classifier** dengan GridSearchCV untuk optimasi hyperparameter

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(random_state=42)
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

best_model = grid_search.best_estimator_
```

- Metode evaluasi
 - Akurasi: 97%
 - ROC AUC Score: 97%
 - Confusion Matrix:
 - True positives: 19.796
 - False positives: 922
 - True negatives: 15.386
 - False negatives: 0
 - Classification Report:

- Precision: 98%
- Recall: 97%
- F1-score: 97%

```

Akurasi Model: 0.97
ROC AUC Score: 0.97
Confusion Matrix:
[[15386  922]
 [   0 19796]]
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	0.94	0.97	16308
1	0.96	1.00	0.98	19796
accuracy			0.97	36104
macro avg	0.98	0.97	0.97	36104
weighted avg	0.98	0.97	0.97	36104

- MAPE Model: 8.90%

Hasil

- **Model Random Forest** menunjukkan akurasi tinggi dalam memprediksi keterlambatan pengiriman, dengan F1-score 97%. dan tidak ada false negatives, yang berarti model dapat mendeteksi semua keterlambatan dengan baik
- **Analisis hasil dan insight**
 - Faktor utama yang mempengaruhi keterlambatan adalah perbedaan antara **days for shipping** dan **days for shipment**
 - Mode pengiriman juga mempengaruhi risiko keterlambatan, khususnya pada metode pengiriman yang lebih lambat

Kesimpulan

Analisis menunjukkan bahwa faktor utama resiko keterlambatan adalah perbedaan antara waktu pengiriman aktual dan yang dijadwalkan, serta jenis pengiriman yang digunakan. Model Random Forest memberikan performa tinggi dengan akurasi **97%**, sehingga dapat digunakan untuk membantu perusahaan dalam mengoptimalkan perencanaan pengiriman dan mengurangi risiko keterlambatan.

Saran

Untuk pengembangan selanjutnya, analisis bisa diperluas dengan menambahkan faktor (atribut) seperti cuaca dan kondisi lalu lintas yang bisa saja mempengaruhi keterlambatan pengiriman.